



Auto uning of price prediction models for high-frequency trading via reinforcement learning

Weipeng Zhang^a, Ning Zhang^{a,*}, Junchi Yan^a, Guofu Li^b, Xiaokang Yang^c

^a Department of Computer Science and Engineering, and MoE Key Lab of Artificial Intelligence, AI Institute, Shanghai Jiao Tong University, No. 800, Dongchuan Road, Shanghai, China

^b Ping An Asset Management Co., Ltd. No. 1333, Lujiazui Road, Shanghai, China

^c MoE Key Lab of Artificial Intelligence, AI Institute, Shanghai Jiao Tong University, No. 800, Dongchuan Road, Shanghai, China

ARTICLE INFO

Article history:

Received 30 June 2020

Revised 14 December 2021

Accepted 16 January 2022

Available online 18 January 2022

Keywords:

High-frequency trading

Inverse reinforcement learning

Parameter optimization

Multi-armed bandit

ABSTRACT

In this paper, we propose an online model optimization algorithm based on reinforcement learning for quantitative trading. The combination of prediction model and trading policy is the most commonly used framework in practical quantitative trading. Integrated with machine learning methods, this framework brings huge profits to quantified companies. In the framework, the prediction model is used to predict future trading price trend, and the trading policy is used to determine the price and number of orders. Even though, the shortcomings of machine learning models are obvious, mainly are, (1) Slow prediction speed. Huge human-craft features and model computing cost much time, which is ten times of pure trading policy without model. (2) Poor generalization. This kind of models can hardly adapt to market data in each period, because market traders will change time to time at micro level, thus the distribution of market data will change. But current model is trained on a long period dataset, it achieves best effect at average, but can not adapt to different market at each period. To address this problem, we propose a novel online model optimization algorithm. A light model library will be constructed. Each light model in this library corresponds to a different market distribution. By devising the appropriate reward function via inverse reinforcement learning algorithm, the algorithm can accurately estimate the profits of each model. Then the model can be selected automatically in real-time trading, so that the trading policies can automatically adapt to changes in trading market, overcoming previous shortcoming of manually updating model and slow prediction speed. Experimental results show that the proposed algorithm achieves state-of-the-art performance on China Commodity Futures Market Data.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

In recent years, the quantitative trading market has been fully developed. The related technical indicators and trading policies emerge in endlessly. The framework that combines models and trading policies has been widely used in real trading and has brought huge profits to enterprise.

At the same time, with the popularity of machine learning technology, machine learning techniques have achieved great success in a variety of tasks, such as computer vision [1–3], natural language processing [4–6] and speech recognition [7–9]. Machine learning techniques have attracted lots of attention from various fields, and it is no exception in the quantitative trading field. Espe-

cially, the work of machine learning for price prediction, which has greatly improved the profitability of quantitative trading. Utilizing machine learning algorithms [10] to model multidimensional factors is more powerful than the traditional methods which can only consider a small number of factors. In high-frequency trading, trading agent can close position shortly (usually one second), then a large number of trading volume is generated, which is usually several millions of transaction per day. Machine learning algorithms is suitable to handle such a large amount of data.

The success of reinforcement learning [11,12], which is an important technique of machine learning, have been extensively demonstrated in a variety of tasks [13], such as traffic signal control [14], game playing [15,16], robot navigation [17], neural architecture search [18]. The great success of Go [19–21] has made researchers aware of the powerful capabilities of reinforcement learning. Thus some researchers have been committed to the exploration of automatic trading robots.

* Corresponding author.

E-mail addresses: nightcrawler@sjtu.edu.cn (W. Zhang), ningz@sjtu.edu.cn (N. Zhang), yanjunchi@sjtu.edu.cn (J. Yan), liguofu149@pingan.com.cn (G. Li), xkyang@sjtu.edu.cn (X. Yang).

As we all know, the function of a quantitative trading program is to issue a trading order to the exchange after performing a series of complex calculations based on trading logic whenever market conditions arrive.

We can regard the order as an action, which contains three main elements: direction (buying or selling), price, and volume. Due to the complexity of the problem, training a network in the quite short term then output the action at each moment is too difficult. Therefore, the general trading framework is that using machine learning models to predict the future price fluctuations and the trading direction (buy or sell). Then trading strategies are manually written to determine the order price and order amount, so as to obtain a complete quantitative trading program.

The most serious problem with this framework is that the order price and order volume are determined by trading strategy written based on human experience. The advantage of this approach is simple and direct, while the disadvantage is that it can not automatically adapt to trading market conditions. Can we use reinforcement learning algorithms to automatically adjust different trading strategies to indirectly realize automatic trading robots? The answer is true. In the above trading framework, if we have a machine learning model library [22], changes in these model library will have a quite significant impact on real profit, so we can use reinforcement learning algorithms to automatically select optimal model to automatically adapt to trading market conditions. Also we can regard the model library as a directed graph [23,24], since one type of trading market has some probability to evolve into another type of trading market, and this graph relation [25] can help to reduce the exploration in action space.

In this paper, we propose an online optimization algorithm for prediction model of trading strategies based on reinforcement learning. The main principles are:

- Choosing one light model equals to taking one action from the view of reinforcement learning, so that the evaluation of each set of hyper-parameters becomes the reward score of the actions in reinforcement learning.
- Devise a reward function which is suitable for the trading scenario based on inverse reinforcement learning [26,27], providing a reference for dynamic adjustment.
- Propose a powerful action selection algorithm (UCB (Upper Confidence Bound) with online back-testing sub system, which can give approximate reward for each action), making it suitable for the application scenario of quantitative trading.

In the experimental part, based on the data of the Chinese Commodity Futures Market, we design a sufficient comparative experiment, using the annualized average profit and the Sharp index to evaluate the performance. The experimental results show that our algorithm greatly improves the profitability of trading policies and the intelligence and automation of quantitative trading procedures.

In summary, our main contributions are as follows:

- A light model library is constructed. And different light models are trained for different data distributions of market.
- We devise a novel reward function based on inverse reinforcement learning, which can accurately estimate the profits of each trading order.
- We devise an algorithm for dynamically selecting model based on reinforcement learning, including an improved Upper Confidence Bound algorithm with a powerful back-testing sub system.
- The proposed system has been run on real-world high-frequency trading in China Commodity Future market, and it shows promising performance. To the best of our knowledge, this system is the first one based on reinforcement learning that deployed in online trading system.

In summary, this paper achieves a major advance in the field of quantitative trading robots, and brings a new idea for quantitative trading researching. Since we bring a novel and practical reinforcement learning method for high-frequency trading task, while most of existing reinforcement learning paper have no practicability. They usually focus on predict next action directly via reinforcement learning, but real trading is much more complicated. In this paper we give a progressive way to apply reinforcement learning algorithm.

The remainder parts of this paper are organized as follows. Section 2 describes the related works. We describe our reinforcement learning based algorithm, and online model optimization algorithm in Section 3. Section 4 details the experimental setup, evaluation metrics, experimental methods, experimental results and analysis, etc. Section 5 concludes this work.

2. Related works

We review the related works in the following aspects: 1) financial trading; 2) upper confidence bound algorithm; 3) inverse reinforcement learning; 4) hyper-parameter optimization.

2.1. Financial trading

Financial trading is a time-honored topic that has attracted extensive attention from modern artificial intelligence and investment industry. At present, more and more machine learning methods [28], especially reinforcement learning techniques [29–33], are applied to automated financial trading program [34]. Reinforcement learning techniques learn by interacting with environment and gradually improve its performance [35] by trial-and-error, thereby greatly improving the performance of the trading program. Many of these studies focus on the stock market [32,36], such as forecasting price fluctuations of stock market [37] and devising optimal stock trading strategies [38]. There are some works dedicated to the study of financial portfolio optimization [39,40].

2.2. Upper confidence bound algorithm

UCB (Upper Confidence Bound) algorithm [41] is one of the most commonly used strategies for stochastic multi-armed bandits problem. It is based on the principle of optimism in the face of uncertainty [42,43], which is to choose actions as if the environment is as nice as plausible possible. Further, [44] utilizes a tighter upper confidence bound to bring a sharper regret line and excellent performance for the algorithm. Due to the multi-armed bandits technique gained great visibility, UCB algorithm also frequently appears in various applications. For example, [45] explores the k -Nearest Neighbour UCB algorithm for multi-armed bandits with covariates; [46] accelerates AdaBoost algorithm with UCB algorithm; [47] proposes a novel upper confidence bound algorithm based on the law of the iterated logarithm for multi-armed bandit game in the fixed confidence setting.

2.3. Inverse reinforcement learning

Inverse Reinforcement Learning (IRL) is the problem of extracting the reward function given observed behavior of an expert underlying a Markov decision process (MDP). The initial work on IRL was done by Ng et al. [26], which solved the inverse reinforcement learning problem for moderate-sized discrete and continuous domains. Then, Abbeel and Ng [48] studies a special case that the reward function is expressible as a linear function of known features which is extracted from state space and action space. Ramachandran and Amir [49] poses IRL problem as a Bayesian learning task and provided a theoretical framework and tractable algorithms for Bayesian IRL. Levine et al. [50] regards the reward as a

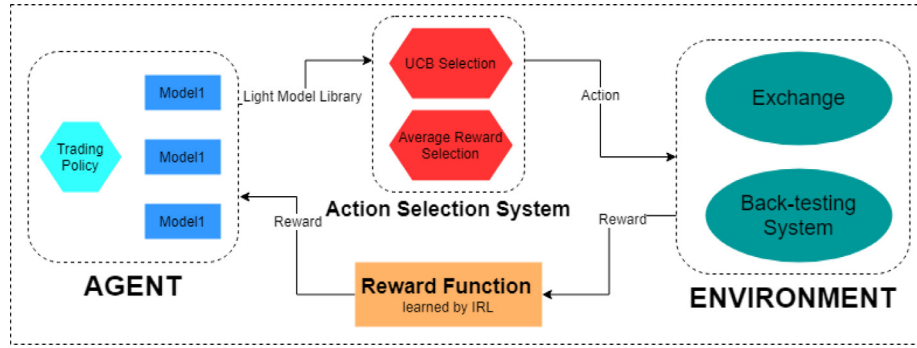


Fig. 1. Reinforcement learning framework for model optimization in high-frequency trading policy. Trading agent is composed of one trading policy and model library with several light models. Trading agent selects a best model at each tick, thus a model stands for an action. Algorithms 2 and 3 describe UCB selection and Average Reward Selection. Trading agent send actions to exchange/back-testing system via action selection algorithm, and receive proper reward (described by Algorithm 1) which can estimate the effect of each action accurately.

non-linear function with Gaussian processes. Using a probabilistic model of a stochastic expert with a GP prior on reward values, the algorithm [50] presented is able to recover both a reward function and the hyper-parameters of a kernel function that describes the structure of the reward. More recent works [51] adopt inverse reinforcement learning for effective and robust learning of mixture temporal point process [52], regarding with deep generative models [53] for temporal event data.

2.4. Hyper-parameter optimization

From the development of machine learning in recent years, changes in the hyper-parameters of a model often have a great impact on the performance of the model. The optimization of model hyper-parameters has gradually evolved into an important research direction [54]. In the field of hyper-parameter optimization, some classic methods have emerged, such as random search [55], evolutionary algorithms [56–58], etc. Specifically in the field of financial transactions, financial trading programs contains a lot of control parameters, which are regarded as hyper-parameters of trading models. The work [59] discusses in details how the parameter choices are made according to the available historical data and the parameters are tuned in order to achieve optimal performance.

Based on above related work, we can list the following differences between our method and the existing works: 1) A light model set is proposed in this paper, while existing works focus on one model which is trained by reinforcement learning framework directly. Since the distribution of market data changes quickly, one model can not always give accurate prediction. Also each light model has high relationship with other models [60], since one light model focus on one type of market distribution, and each type of market distribution has high relationship with each other. 2) Reward function is learned by inverse reinforcement learning algorithm based on short and long expectation, while existing works usually define a fixed and sparse reward [61] function by human experience. 3) Online reward of each action can be determined by backtesting sub system to enhance the effect of UCB selection algorithm, while existing UCB problems can get only one action's reward at each exploration time. 4) The proposed method is applied in real trading to measure the effect, while most existing works only show the performance at backtesting system.

3. The proposed approach

Reinforcement Learning framework of our proposed methods is shown in Fig. 1. Trading agent is composed of machine learning models to predict instrument's price change, and several sub policies. Each policy is designed by human, such as taking policy, mak-

ing policy with AskPrice1 or BidPrice1, making policy with limit order price, etc. As we know, order is composed of direction (buy or sell), price and volume. The direction of order is determined by machine learning model. The price and volume of order are determined by above policies. Then the order is sent by trading agent to environment, which is exchanged in real trading or back-testing system. At each tick, our action is to choose one model for price prediction. Thus the high-frequency trading task is converted to an reinforcement learning problem, which aims to select the optimal action (choose a model).

There are three challenge problems in the proposed framework: 1) Developing light model library for action selection; 2) Proposing proper reward function to measure action. The reward function plays a significant role to judge the effect of each action. 3) Selecting optimal action at each tick. We will explain our proposed methods for these three challenges in following parts.

3.1. Definition and notation

We use M_i for each light model, $i = 1, 2, 3, \dots, N$, thus we have N light models in our trading agent. Machine learning model M is trained by history data. As we mentioned in above, the action A_j in our reinforcement learning framework is corresponding to choosing a light model M_j , suppose we have K actions, $j = 1, 2, 3, \dots, K$.

$$A_j = \{M_j\}, j = 1, 2, 3, \dots, K \quad (1)$$

In general, high-frequency trading will close position within current day, so one day is a complete optimal period for reinforcement learning. We can use our methods at the start time within one day to optimize model for each tick, improve the profit in real trading. Thus, our problem can be formulated as selecting optimal action A_j at each trading tick within one day.

Table 1 shows the main notations of this paper, while tick is a common word in high-frequency trading. In China Commodity Future market, data is published in snapshot for every half second (although Dalian Commodity Exchange published every quarter second). A tick is 0.5 s, which is the minimum time unit in trading. Thus we usually say that there are two ticks per second, and there are 41,400 ticks in one trading day, since the trading time is 9:00–10:15, 10:15–11:30, 13:30–15:00, 21:00–23:00.

3.2. Light model library

We construct model library in this section, as we mentioned in above, model library is corresponding to action space in reinforcement learning framework. Feature and time are two aspects for model library constructing.

Table 1

The main notations used in the paper, and those with * denote the hyper-parameters in our approach.

Notation	Description
M	Model library
m^*	Each light model
A	Action in RL framework, means light model in this paper
O	Market order which sent to exchange for trading
E	Order expectation, means difference between traded tick midprice and future tick midprice
T	Tick period
t	Minimum time unit of market data publish
t^t	Tick number from open time when order is traded
m_{price}	Midprice, means $(A_{price} + B_{price})/2$
A_{price}	The best sell price in order book
B_{price}	The best buy price in order book
R	Reward function
D	Train Set

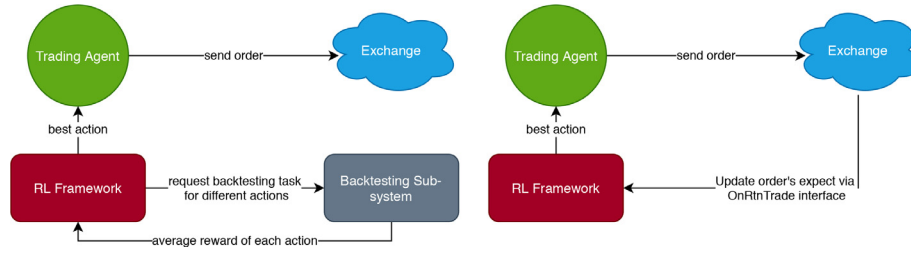


Fig. 2. Reinforcement learning system for real trading. Left figure is about FMATO-OE-ARS. Back-testing sub-system is implemented for back-testing each parameter set when live trading. The actions can be measured by its average reward from back-testing sub-system. Right is FMATO-OE-UCBS, average reward and visited time from live trading of each action are recorded for selection via UCB algorithm.

Table 2

Feature set for model training. Price and volume data are main features in high-frequency trading.

Feature Set	Description
Price-related Feature	Price change, Move average price, Var of price, Speed of Price, etc
Volume-related Feature	Volume change, Move average volume, Var of volume, Speed of volume, etc
Price-Volume-related Feature	Multiply of price diff and volume, Division of price diff and volume, etc

The light models in this paper are designed as decision trees or linear regression models. Each model is trained using different feature set [62] with different time period.

In China Commodity Future market, the distribution of market data changes weekly in usual. Thus we can select data weekly to train the light models.

Table 2 shows different feature sets for model training. The model library will be updated every week.

3.3. Expectation-based reward function

Reward function plays a significant role in reinforcement learning for measuring actions. In this paper, we use expectation in reward function, which is defined as

$$E(T) = m_{price}^{t^t+T} - m_{price}^{t^t} \quad (2)$$

where T is the time period, if we focus on optimizing expectation within short period, T can be set as 10, 20, or 30 ticks. If we focus on long period, T can be 90, 270, 810, or 2430 ticks, ..., etc.

In production environment, most trading system just focus on short period expectation, which has visible ceiling of trading profit. But there are more profit when holding a position with longer period. It seems that the smarter method is combining the short and long period expectation. However the future trading is a highly dynamic process. The distribution of market data is always changed, and has much difference with previous data. Thus online learning is a more novel and accurate approach for combination with short and long period expectation. In this paper, our reward function will consider multi-scale period expectations, including 10 ticks, 30

ticks, 90 ticks, ..., the longest period will be more than one hour. Thus the reward function is:

$$E(O) = \sum w_i * E_T, T = [10, 30, 90, \dots] \quad (3)$$

where $\sum w_i = 1, i = 1, 2, \dots, K$.

We use inverse reinforcement learning to optimize the reward function. Our reward function is linear function, $E(O) = w * u$, where u is the feature expectation,

$$u = [E_{10}, E_{30}, E_{90}, \dots]^T \quad (4)$$

Algorithm 1 describes the detail of parameter learning for reward function.

Algorithm 1 IRL for reward function of financial prediction model.

input: Training set D ;

output: Best parameter w_* of reward function;

- 1: Analysis back-testing result on training set D , get expert policy;
- 2: init a random policy p_j , compute the initial feature expectation u_j ;
- 3: optimize parameter w_i to maximize the expectation difference $error_j$ between expert policy and p_j ;
- 4: if $error_j < \epsilon$, then stop and get best parameter $w_* = w_i$;
- 5: based on reward function $R(w_i)$, get current optimal policy p_{j+1} by applying parameter optimization algorithm;
- 6: compute the feature expectation u_{j+1} ;
- 7: set $j = j + 1$, loop again from step 3;

To better evaluate the reward function, we consider two types of expectations, model prediction expectation and order traded expectation. Model prediction expectation describes the upper limit of expectation which we can get from trading. It provides a more accurate way to measure the effect of model than accuracy and mean square error. In addition, we usually pay more attention on the expectation which sorted above one-in-one-thousand by model prediction value, because in real trading, these signal brings more credible trading opportunity. Order traded expectation describes the real expectation which we can get from trading. It provides an evaluation method for both model and policy, since each trading order is determined by both the model and the policy. We not only pay attention on short period expectation of traded order, also long period expectation. Thus these two types of expectations provide different viewpoints for the evaluation of model and policy.

3.3.1. Model prediction expectation

In this case, the calculation of reward function $R(a)$ will be based on model prediction expectation. For example, the trading agent selects a light model at current tick, then the model gives a prediction value about future price change. We can record its expectation $E(T)$ as the difference between the midprice at prediction tick and the midprice at future tick.

3.3.2. Order traded expectation

In this case, we will design reward function $R(a)$ based on each order's expectation after traded. For example, the trading agent sends an order $O(100.00, 1, \text{buy})$, which means price 100.00, volume 1, direction buy, to exchange or back-testing system. When this order is traded, we can record its expectation $E(T)$ as the difference between midprice at traded tick and midprice at future tick. For example, if one order is traded at 09:01:10 with price 99, and after 10 ticks, time is 09:01:15 while price is 102, thus $E(10) = 102 - 99 = 3$.

3.4. Model optimization

After obtaining optimized reward function $E_*(O)$, the reward of each action $E(a)$ can be determined by the expectations of all orders sent by this action within a certain trading period.

$$E(a) = \sum_{i \in p} E_*(O_i) / M \quad (5)$$

where O_i is an order sent by this action within a fixed trading period p , M is the number of orders in this period.

Then the Upper Confidence Bound (UCB) [41] algorithm can be used to select action at each trading tick.

3.4.1. UCB selection

UCB is widely used in dynamic optimization task, which aims to find the best bandit with maximum expectation. Eq. (6) describes each component of UCB score of each action.

$$Q(a) = W(a) + C * \sqrt{2 * \ln N / n(a)} \quad (6)$$

where $Q(a)$ is UCB score of each action, $W(a)$ is average reward, $n(a)$ is explored times, C is a constant coefficient. The first term stands for exploitation, while the second term stands for exploration. Algorithm 2 shows the detailed step for model selection by bandit learning.

3.4.2. Average reward selection

In traditional reinforcement learning problem, only one action can get reward feedback at the same time. In our trading agent, we develop a sub-system for back-testing a specify period in real trading. Thus we can get an extra reward of all other actions at the same time by back-testing sub-system. Then another algorithm is

Algorithm 2 UCB for financial prediction model selection.

input: Model set A , Reward function $R(a)$

output: Optimal model a_t at each trading tick t ;

- 1: Init $W(a_i) = 0.0, n(a_i) = 0$;
 - 2: Select model a_t^i with highest UCB score;
 - 3: Trading with model a_t^i , then record the reward of each order in fixed period (e.g. 5 minutes);
 - 4: Update $W(a_i)$ and $n(a_i)$;
 - 5: loop again from step 2;
-

Algorithm 3 Average reward for financial prediction model selection.

input: Model set A , Reward function $R(a)$;

output: Optimal model a_t at each trading tick t ;

- 1: Init $W(a_i) = 0.0$;
 - 2: Select model a_t^i with highest score $W(a_i)$;
 - 3: Trading with model a_t^i , then record the reward of each order in a fixed period (e.g. 5 minutes);
 - 4: back-testing all other models a_t^j in a fixed period (e.g. 30 minutes) before current tick, where $j \neq i$;
 - 5: Update $W(a_i)$ for all models
 - 6: loop again from step 2;
-

using all reward from back-testing sub-system, called average reward selection (ARS). Algorithm 3 describes the model optimization in detail. Back-testing sub-system plays a significant role for action reward estimation.

4. Experiments

In this section, we design extensive and detailed experiments to prove effectiveness and originality of our algorithm FMATO. Real trading data of China Commodity Future market is used in experiments. Because in China derivative market, Future market is the main platform for high-frequency trading. $T + 0$ trading principle is used in China Commodity Future market. Here we call commodity future product as instrument. For example, instrument $i2005$ means the future product of iron ore, which is expired at May of 2020, and instrument $j2005$ means the future product of coke, which is expired at May of 2020. The property of high-frequency means that there are more data, 4 snapshots of market data in one second, including order book, trade volume and trade turnover. Here each snapshot is named as tick. Thus real trading agent (e.g., human, trading program) can close position shortly (usually one second), then a large number of trading volume (usually several millions of transaction per day) is generated. And machine learning algorithms can play a significant role in high-frequency trading.

4.1. Dataset

For the adequacy of experiments, we prepare datasets from two aspects:

- Two different future instruments with highest turnover, dominant instrument of $j2005$ and $i2005$.
- Three different sizes of market datasets, which includes trading records of three months, six months, twelve months, respectively.

Table 3 shows the information of each instrument, including multiple, min price unit, etc. These information have dominating effects for each instrument's trading volume and profitable space.

Table 3
Information of dominant instrument.

Instrument	Min price unit	Multiple	Commission	Profit per tick	Mean market order volume#
j2005	0.5	100	0.00006	30 rmb	30
i2005	0.5	100	0.0001	38 rmb	1000

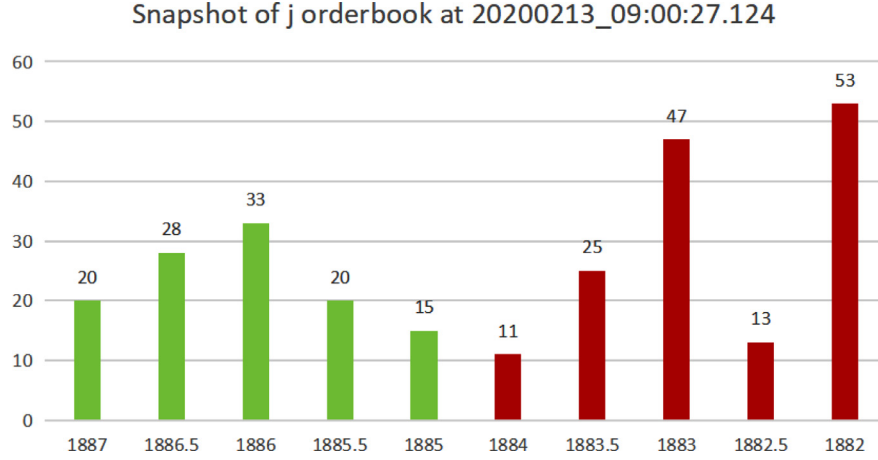


Fig. 3. Sample limit order book snapshot of instrument j2005. Green bars represent ask price and ask volume, red bars denote bid price and bid volume. Horizontal axis stands for price, vertical axis stands for volume. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

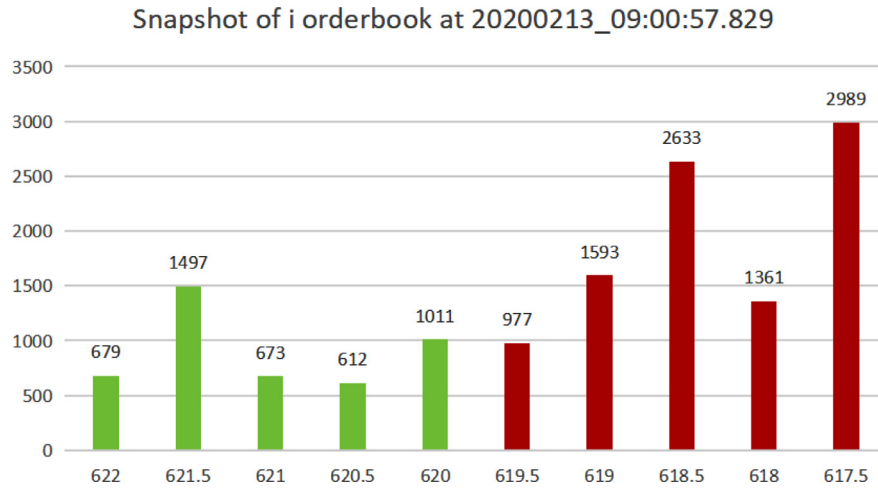


Fig. 4. Sample limit order book snapshot of instrument i2005. Green bars represent ask price and ask volume, red bars denote bid price and bid volume. Horizontal axis stands for price, vertical axis stands for volume. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Figs 3 and 4 show the snapshot of instruments j2005 and i2005. Most high-frequency traders pay much attention on these two instruments, since they have much profit per tick. Thus these two instruments have most trading volume and turnover. In addition, they are the two most representative instruments. Taking and making policy are two most typical policy in high-frequency trading. Taking policy have better profit on instruments which have small volume in order book (usually 10 to 50), since trading agent can close position quickly. While making policy have better profit on instruments which have large volume in order book (usually 500 to 1000), since making orders will have high expectation after traded generally.

These experimental results on these two instruments datasets can verify the strong applicability and generality of our proposed approach.

4.2. Results of model optimization by FMATO

Current trading algorithm (Model + TradingPolicy) is designed as baseline to compare with our method FMATO.

- **Model+TradingPolicy:** Single machine learning model (usually we use GBDT or RNN), which is trained on history dataset. Trading policies are developed with human knowledge, like some taking policies, and making policies.

Based on our method, we deploy four different algorithms as follows:

- **FMATO-ME-UCBS:** use model prediction expectation for reward function, and use UCB selection method for model optimization.

Table 4

Model optimization: comparison results of baseline and our methods FMATO at instrument j2005.

Method	Experiment setting	Close profit	Commission
Baseline1	Single Model	308,900	260,400
FMATO-ME-UCBS	Model Prediction expectation, UCB selection	460,100	259,414
FMATO-OE-UCBS	Order Traded expectation, UCB selection	493,600	249,876
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	453,800	233,875
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	508,200	253,529

Table 5

Model optimization: comparison results of baseline and our methods FMATO at instrument i2005.

Method	Experiment setting	Close profit	Commission
Baseline1	Single Model	-3400	20,464
FMATO-ME-UCBS	Model Prediction expectation, UCB selection	178,300	93,872
FMATO-OE-UCBS	Order Traded expectation, UCB selection	256,400	193,527
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	209,200	110,358
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	244,900	150,028

- FMATO-OE-UCBS: use order traded expectation for reward function, and use UCB selection method for model optimization.
- FMATO-ME-ARS: use model prediction expectation for reward function, and use average reward selection method for model optimization.
- FMATO-OE-ARS: use order traded expectation for reward function, and use average reward selection method for model optimization.

Tables 4 and 5 show the results of all these algorithms on two datasets, instrument j2005 with three months data, and instrument i2005 with three months. All experiments use the last one month as testing set, the others as training set. RMB is the unit of profit and commission in each table.

In Table 4, the commission of *Baseline1* is almost equal with our method, but the profit of our method is larger than *Baseline1*. The algorithms *FMATO-OE-UCBS* and *FMATO-OE-ARS* Fig. 2 get the most profit, which shows that the order traded expectation is better than the model prediction expectation for instrument j2005. In Table 5, the commission of *Baseline1* is far less than our methods, which means that our method find much more trading signal for instrument i2005. The profit of *Baseline1* is negative, while our method performs better and get much more profit.

4.3. Time cost for model optimization by FMATO

In high-frequency trading, speed is one of the most significant factor. The faster trading order is sent to exchange after receiving the market data, the more trading profit. As we mentioned above, single model is trained by several hundreds of decision trees or several layers of deep neural network, and use more than one hundred features. This will cost the most time in real trading per tick, and slow down the order speed to exchange, thus reduce much profit in real trading.

In this section we will compare the time cost between single model policy and our FMATO method. The time cost is defined as the time between trading program received market data and sent order to exchange per tick. And these experiments are all run on machine with Intel Xeon Gold 6154 CPU 3.00 GHz.

Tables 6 and 7 show the results of time cost. Our method FMATO reduces time cost significantly, just 10 percent of time cost of baseline method. This is due to the designed light machine learning models, the combination of simple model library and effective model optimization algorithm. The algorithms with average reward selection cost more time than UCB selection, since they use back-testing sub systems to compute the approximate expectation of each model.

4.4. Result for reward function

In this section we design several baselines to compare with our two algorithms. They are listed as follows:

- $E(10 \text{ tick})$: Future 10 tick expectation after traded.
- $E(\text{Current tick})$: Expectation between traded tick and current tick.
- Linear Combine: Linear combination of short and long period expectation.

Our four algorithms show details in Table 7. UCB selection method is used for model optimization. Tables 8 and 9 show the results of different reward functions. Profit and commission are set as metrics. Larger profit means better reward function, also larger value of division between profit and commission means better reward function. In Table 8, the commission of each algorithms is almost equal with others, while profits of our algorithms are much better than baseline algorithms. *Baseline1* uses a fixed period as expectation, which just focus on short period expectation, while long period is also important for expectation evaluation. *Baseline2* describes real-time expectation, which can not evaluate the accurate expectation, since if an order's expectation is very high or low, it may be the luck. Thus this reward function is very stochastic for order evaluation, not reliable. *Baseline3* is more reliable than above baselines, since it combines short and long period expectation with manually setting weight.

4.5. UCB selection vs. model ensemble

Ensemble learning is an effective algorithm which combines each model's advantages, also can achieve the single model's prediction speed, thus have advantage both on speed and prediction accuracy.

We design several baselines to compare with our two algorithms, including:

- Single: Single best model in model library,
- Ensemble1: Average each model's prediction,
- Ensemble2: Linear combination of each model's prediction, use model accuracy as weight.

Our four algorithms show detail above. UCB selection method is used for model optimization. Tables 10 and 11 show the results of different model optimization methods. Prediction expectation and accuracy are set as metrics. Larger expectation or accuracy means better optimization method. In Table 10, our proposed algorithms perform much better than baselines, especially the algorithms using order traded expectation. *Baseline2* (*Ensemble1*) takes the average ensemble method, reduces the variance, but may increase bias.

Table 6

Time cost: the time cost comparison of baseline and our methods FMATO at instrument j2005.

Method	Experiment setting	Time cost per tick
Baseline1	Single Model	200 us
FMATO-ME-UCBS	Model Prediction expectation, UCB selection	20 us
FMATO-OE-UCBS	Order Traded expectation, UCB selection	19 us
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	24 us
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	23 us

Table 7

Time cost: the time cost comparison of baseline and our methods FMATO at instrument i2005.

Method	Experiment setting	Time cost per tick
Baseline1	Single Model	210 us
FMATO-ME-UCBS	Model Prediction expectation, UCB selection	21 us
FMATO-OE-UCBS	Order Traded expectation, UCB selection	20 us
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	25 us
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	24 us

Table 8

Reward function: comparison of three baselines and our methods FMATO at instrument j2005.

Method	Experiment setting	Profit	Commission
Baseline1	E(10tick)	427,500	267,894
Baseline2	E(Current tick)	394,600	273,485
Baseline3	Linear combine	438,300	263,145
FMATO-ME-UCBS	Model prediction expectation, UCB selection	460,100	259,414
FMATO-OE-UCBS	Order traded expectation, UCB selection	493,600	249,876
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	453,800	233,875
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	508,200	253,529

Table 9

Reward function: comparison of three baselines and our methods FMATO at instrument i2005.

Method	Experiment setting	Close profit	Commission
Baseline1	E(10tick)	27,500	42,684
Baseline2	E(Current tick)	-5500	31,076
Baseline3	Linear combine	38,300	33,145
FMATO-ME-UCBS	Model prediction expectation, UCB selection	178,300	93,872
FMATO-OE-UCBS	Order traded expectation, UCB selection	256,400	193,527
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	209,200	110,358
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	244,900	150,028

Table 10

Model optimization: comparison of three baselines and our methods FMATO at instrument j2005.

Method	Experiment setting	Expectation	Accuracy
Baseline1	Single	0.64	62%
Baseline2	Ensemble1	0.74	70%
Baseline3	Ensemble2	0.76	75%
FMATO-ME-UCBS	Model prediction expectation, IRL	0.95	89%
FMATO-OE-UCBS	Order traded expectation, IRL	0.98	92%
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	0.94	88%
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	0.96	93%

Table 11

Model optimization: comparison of three baselines and our methods FMATO at instrument i2005.

Method	Experiment setting	Expectation	Accuracy
Baseline1	Single	0.25	61%
Baseline2	Ensemble1	0.28	68%
Baseline3	Ensemble2	0.30	72%
FMATO-ME-UCBS	Model prediction expectation, IRL	0.35	78%
FMATO-OE-UCBS	Order traded expectation, IRL	0.38	83%
FMATO-ME-ARS	Model Prediction expectation, Average Reward selection	0.32	78%
FMATO-OE-ARS	Order Traded expectation, Average Reward selection	0.36	84%

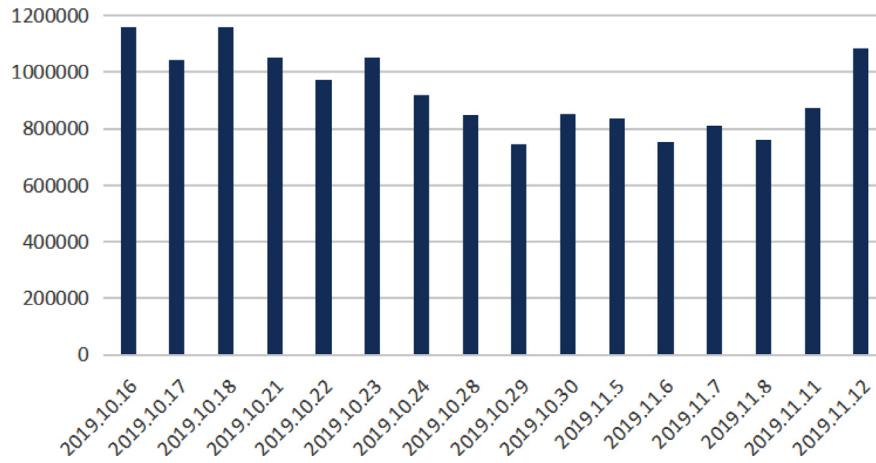


Fig. 5. Daily trading volume of instrument i2001. X-axis stands for trading day, Y-axis stands for trading volume.

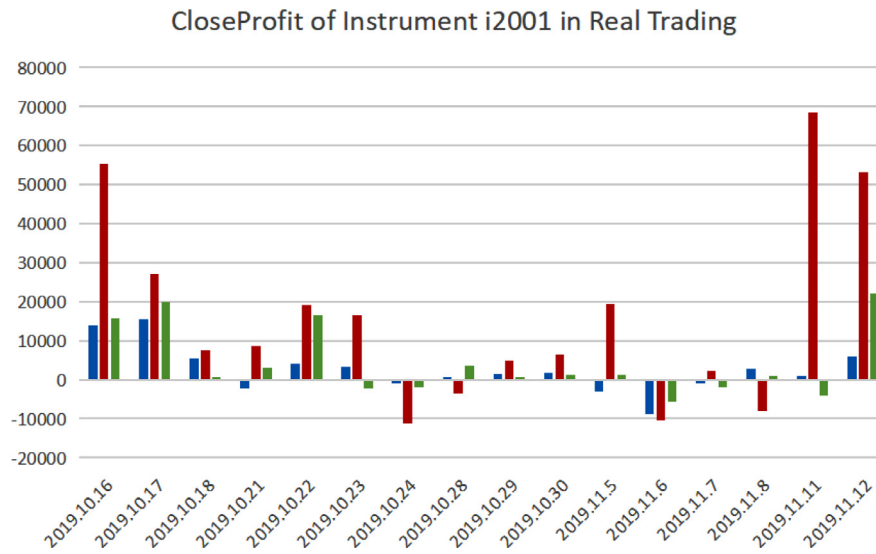


Fig. 6. Daily close profit comparison of our proposed three algorithms. Red line stands for algorithm FMATO-OE-ARS, green line stands for FMATO-OE-UCBS, blue line stands for Baseline. X-axis stands for trading day. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

It performs better than Baseline1 (*Single*), even 8% improvement on accuracy. Baseline3 (*Ensemble2*) performs even better, since it takes each model's accuracy as weight.

4.6. Profit curve of real trading

In addition to conducting experiments on history dataset, we also deploy our two kinds of model optimization into real trading at China Future market.

We prepare two future accounts for this experiment, and run one algorithms at each account for a month since 2019-10-16. 1) FMATO + Model Prediction expectation + UCB Selection, 2) FMATO + Order Traded expectation + UCB Selection.

For each account, we recorded trading commission and close profit. Thus two viewpoints can be measured for three algorithms, one is just the value of close profit, larger value means larger profit, other one is profit multiple (close profit / trading commission), larger multiple means better trading policy.

Fig. 5 shows daily trading volume of instrument i2001. In general, larger trading volume will bring more trading signal. Then the model optimization effect will be more better, such as 2019-10-16, 2019-11-12.

4.6.1. Close profit

From Fig. 6, FMATO-OE-ARS algorithm gets best value of close profit at most trading days. FMATO-OE-UCBS algorithm also gets much better value of close profit than *Baseline(singlemodel)*, but worse than FMATO-OE-ARS at some trading days.

4.6.2. Profit multiple

From Fig. 7, FMATO-OE-ARS algorithm also gets best value of profit multiple at most trading days. And at some trading days with small trading volume, FMATO-OE-UCBS also plays well, such as 2019-10-29, 2019-11-08.

4.7. Case study

Figs. 8 and 9 show the price distributions in one trading day. Our online model optimization algorithm help to bring much profit for trading agent. In Fig. 8, our algorithm will adjust models to send more making policy orders in period which the first blue circle pointed. And in second blue circle, models will be adjusted to taking policy module, so as the third blue circle. The volatility of price in last two blue circle are very high, while the first blue circle is low. Thus our online model optimization algorithm can adapt

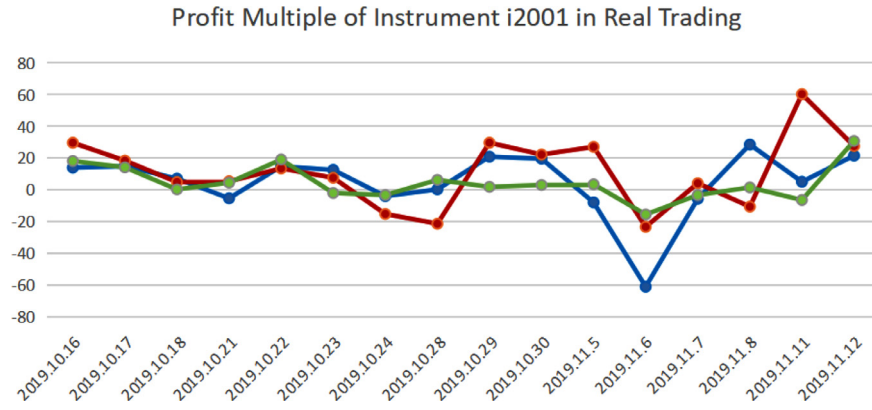


Fig. 7. Daily profit multiple comparison of our proposed three algorithms. Red line stands for algorithm FMATO-OE-ARS, green line stands for FMATO-OE-UCBS, blue line stands for Baseline. X-axis stands for trading day. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

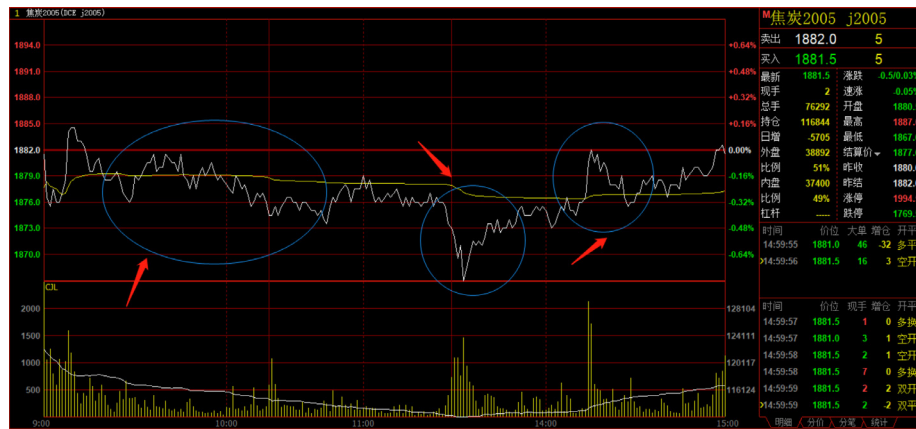


Fig. 8. Snapshot of price distribution chart of instrument j2005

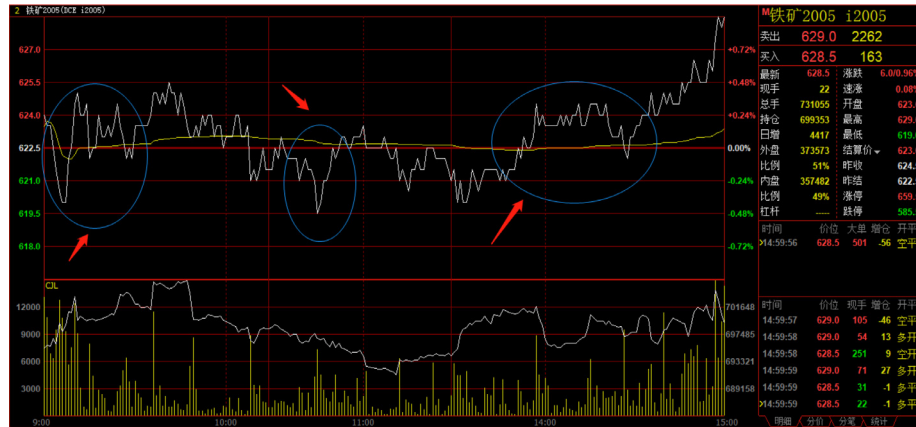


Fig. 9. Snapshot of price distribution chart of instrument i2005

to market very quickly, and select the proper model on different market condition. Fig. 9 is also a powerful case, especially in second blue circle. Our algorithm can optimize model timely to avoid negative profit of making policy module, even taking orders will be sent to grab a great chance.

5. Conclusion and future work

In this paper, we have presented a novel and effective approach for model online optimization of high-frequency trading agent in

China Commodity Future market. In fact, the proposed approach has strong applicability and generality. It can be used in any secondary market which applied $T + 0$ trading principle. Inverse Reinforcement Learning plays a significant role for reward function design, especially for the proper combination of short period expectation and long period. Also a powerful back-testing sub-system is implemented to provide online expectation in real trading. Extensive experiments are deployed on two most representative instruments j and i , which show promising profit improvement in real trading for one month. In future, we will continue the research on

trading robot, not only the model optimization of trading policy, but also the optimization of policy module by reinforcement learning, and exploration about new sub policy module.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

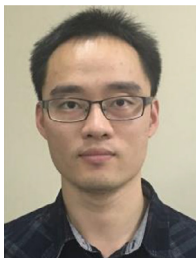
This work was partially supported by Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102), and NSFC (61972250, U19B2035).

References

- [1] R. K. Sinha, R. Pandey, R. Pattnaik, Deep learning for computer vision tasks: a review, *CoRR abs/1804.03928* (2018).
- [2] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, *ICLR* (2015).
- [3] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *CVPR*, 2016, pp. 770–778.
- [4] M. Collins, Tutorial: machine learning methods in natural language processing, in: *Learning Theory and Kernel Machines*, Springer, 2003, p. 655.
- [5] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, *NAACL-HLT*, 2019.
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: *NIPS*, 2017, pp. 5998–6008.
- [7] L. Deng, G. Hinton, B. Kingsbury, New types of deep neural network learning for speech recognition and related applications: An overview, in: *ICASSP*, IEEE, 2013, pp. 8599–8603.
- [8] D. Amodei, S. Ananthanarayanan, R. Anubhai, J. Bai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, Q. Cheng, G. Chen, et al., Deep speech 2: end-to-end speech recognition in english and mandarin, in: *ICML*, 2016, pp. 173–182.
- [9] L. Deng, X. Li, Machine learning paradigms for speech recognition: an overview, *IEEE Trans. Audio, Speech, Lang. Process.* 21 (5) (2013) 1060–1089.
- [10] B. Vanstone, C. Tan, A survey of the application of soft computing to investment and financial trading, *Pattern Recognit.* (2003) 211–216.
- [11] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: a survey, *JAIR* 4 (1996) 237–285.
- [12] R.S. Sutton, A.G. Barto, et al., *Introduction to Reinforcement Learning*, vol. 135, MIT Press Cambridge, 1998.
- [13] R.H. Crites, A.G. Barto, Improving elevator performance using reinforcement learning, in: *NIPS*, 1996, pp. 1017–1023.
- [14] I. Arel, C. Liu, T. Urbanik, A.G. Kohls, Reinforcement learning-based multi-agent system for network traffic signal control, *IET Intell. Transp. Syst.* 4 (2) (2010) 128–135.
- [15] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [16] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing Atari with deep reinforcement learning, *CoRR* (2013).
- [17] H.R. Beom, H.S. Cho, A sensor-based navigation for a mobile robot using fuzzy logic and reinforcement learning, *IEEE Trans. Syst., Man, Cybern.* 25 (3) (1995) 464–477.
- [18] B. Zoph, Q.V. Le, Neural architecture search with reinforcement learning, *CoRR* (2016).
- [19] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484.
- [20] D. Silver, R.S. Sutton, M. Müller, Reinforcement learning of local shape in the game of go, in: *IJCAI*, vol. 7, 2007, pp. 1053–1058.
- [21] B. Bouzy, G. Chaslot, Monte-carlo go reinforcement learning experiments, in: *2006 IEEE Symposium on Computational Intelligence and Games*, IEEE, 2006, pp. 187–194.
- [22] D. Cheng, F. Yang, S. Xiang, J. Liu, Financial time series forecasting with multi-modality graph neural network, *Pattern Recognit.* 121 (2022) 108218.
- [23] T. Yin, C. Liu, F. Ding, Z. Feng, B. Yuan, N. Zhang, Graph-based stock correlation and prediction for high-frequency trading systems, *Pattern Recognit.* 122 (2022) 108209.
- [24] D. Chen, X. Guo, J. Wang, J. Liu, Z. Zhang, E.R. Hancock, Thermodynamic motif analysis for directed stock market networks, *Pattern Recognit.* 114 (2021) 107872.
- [25] J. Wang, X. Guo, W. Li, X. Wu, Z. Zhang, E.R. Hancock, Statistical mechanical analysis for unweighted and weighted stock market networks, *Pattern Recognit.* 120 (2021) 108123.
- [26] A.Y. Ng, S.J. Russell, et al., Algorithms for inverse reinforcement learning, in: *ICML*, vol. 1, 2000, pp. 663–670.
- [27] A. Boularias, J. Kober, J. Peters, Relative entropy inverse reinforcement learning, in: *AISTATS*, 2011, pp. 182–189.
- [28] Z. Li, H. Xie, G. Xu, Q. Li, M. Leng, C. Zhou, Towards purchase prediction: a transaction-based setting and a graph-based method leveraging price information, *Pattern Recognit.* 113 (2021) 107824.
- [29] Y. Deng, F. Bao, Y. Kong, Z. Ren, Q. Dai, Deep direct reinforcement learning for financial signal representation and trading, *IEEE Trans. Neural Netw. Learn. Syst.* 28 (3) (2016) 653–664.
- [30] Z. Jiang, D. Xu, J. Liang, A deep reinforcement learning framework for the financial portfolio management problem, *CoRR* (2017).
- [31] K.S. Zarkias, N. Passalis, A. Tsantekidis, A. Tefas, Deep reinforcement learning for financial trading using price trailing, in: *ICASSP*, IEEE, 2019, pp. 3067–3071.
- [32] Z. Xiong, X.-Y. Liu, S. Zhong, H. Yang, A. Walid, Practical deep reinforcement learning approach for stock trading, *CoRR* (2018).
- [33] G. Li, J.J. Jung, Dynamic relationship identification for abnormality detection on financial time series, *Pattern Recognit. Lett.* 145 (2021) 194–199.
- [34] L. Xu, L. Cui, T. Weise, X. Li, Z. Wu, F. Nie, E. Chen, Y. Tang, Semi-supervised multi-layer convolution kernel learning in credit evaluation, *Pattern Recognit.* 120 (2021) 108125.
- [35] L. Wu, Z. Li, H. Zhao, Q. Liu, E. Chen, Estimating fund-raising performance for start-up projects from a market graph perspective, *Pattern Recognit.* 121 (2022) 108204.
- [36] S.F. Yazdani, M.A.A. Murad, N.M. Sharef, Y.P. Singh, A.R.A. Latiff, Sentiment classification of financial news using statistical features, *Int. J. Pattern Recognit. Artif. Intell.* 31 (3) (2017) 1750006:1–1750006:34.
- [37] J.W. Lee, Stock price prediction using reinforcement learning, in: *ISIE 2001. 2001 IEEE International Symposium on Industrial Electronics Proceedings (Cat. No. 01TH8570)*, vol. 1, IEEE, 2001, pp. 690–695.
- [38] J.W. Lee, J. Park, O. Jangmin, J. Lee, E. Hong, A multiagent approach to q-learning for daily stock trading, *IEEE Trans. Syst., Man, Cybern.-Part A* 37 (6) (2007) 864–877.
- [39] Z. Liang, H. Chen, J. Zhu, K. Jiang, Y. Li, Adversarial deep reinforcement learning in portfolio management, *CoRR* (2018).
- [40] Y. Sato, Model-free reinforcement learning for financial portfolios: A brief survey, *CoRR* (2019).
- [41] P. Auer, Using confidence bounds for exploitation-exploration trade-offs, *J. Mach. Learn. Res.* 3 (Nov) (2002) 397–422.
- [42] R. Agrawal, Sample mean based index policies by $O(\log n)$ regret for the multi-armed bandit problem, *Adv. Appl. Probab.* 27 (4) (1995) 1054–1078.
- [43] P. Auer, N. Cesa-Bianchi, P. Fischer, Finite-time analysis of the multiarmed bandit problem, *Mach. Learn.* 47 (2–3) (2002) 235–256.
- [44] A. Garivier, O. Cappé, The KL-UCB algorithm for bounded stochastic bandits and beyond, in: *Proceedings of the 24th annual conference on learning theory*, 2011, pp. 359–376.
- [45] H.W. Reeve, J. Mellor, G. Brown, The k -nearest neighbour UCB algorithm for multi-armed bandits with covariates, *arXiv preprint arXiv:1803.00316* (2018).
- [46] R. Busa-Fekete, B. Kégl, Accelerating adaboost using UCB, in: *KDD-Cup 2009 Competition*, 2009, pp. 111–122.
- [47] K. Jamieson, M. Malloy, R. Nowak, S. Bubeck, lil'UCB: an optimal exploration algorithm for multi-armed bandits, in: *COLT*, 2014, pp. 423–439.
- [48] P. Abbeel, A.Y. Ng, Apprenticeship learning via inverse reinforcement learning, in: *ICML*, 2004, p. 1.
- [49] D. Ramachandran, E. Amir, Bayesian inverse reinforcement learning, in: *IJCAI*, vol. 7, 2007, pp. 2586–2591.
- [50] S. Levine, Z. Popovic, V. Koltun, Nonlinear inverse reinforcement learning with Gaussian processes, in: *NIPS*, 2011, pp. 19–27.
- [51] W. Wu, J.Y. Yan, X. Yang, H. Zha, Discovering temporal patterns for event sequence clustering via policy mixture model, *IEEE Trans. Knowl. Data Eng.* 34 (2020) 573–586.
- [52] S. Xiao, J. Yan, X. Yang, H. Zha, S.M. Chu, Modeling the intensity function of point process via recurrent neural networks, *AAAI*, 2017.
- [53] S. Xiao, M. Farajtabar, X. Ye, J. Yan, L. Song, H. Zha, Wasserstein learning of deep generative point process models, *NIPS*, 2017.
- [54] J.S. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, in: *NIPS*, 2011, pp. 2546–2554.
- [55] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *JMLR* 13 (Feb) (2012) 281–305.
- [56] A.H. Wright, Genetic algorithms for real parameter optimization, in: *Foundations of Genetic Algorithms*, vol. 1, Elsevier, 1991, pp. 205–218.
- [57] Z. Michalewicz, M. Schoenauer, Evolutionary algorithms for constrained parameter optimization problems, *Evol. Comput.* 4 (1) (1996) 1–32.
- [58] T. Bäck, H.-P. Schwefel, An overview of evolutionary algorithms for parameter optimization, *Evol. Comput.* 1 (1) (1993) 1–23.
- [59] O.V. Pictet, M.M. Dacorogna, B. Chopard, M. Oussaidene, R. Schirru, M. Tomassini, et al., Using genetic algorithms for robust optimization in financial applications, *Neural Netw. World* 5 (4) (1995) 573–587.
- [60] S. Feng, C. Xu, Y. Zuo, G. Chen, F. Lin, J. Xiahou, Relation-aware dynamic attributed graph attention network for stocks recommendation, *Pattern Recognit.* 121 (2022) 108119.
- [61] Y. Hua, X. Wang, B. Jin, W. Li, J. Yan, X. He, H. Zha, HMRL: hyper-meta learning for sparse reward reinforcement learning problem, *KDD*, 2021.
- [62] L. Cui, L. Bai, Y. Wang, X. Jin, E.R. Hancock, Internet financing credit risk evaluation using multiple structural interacting elastic net feature selection, *Pattern Recognit.* 114 (2021) 107835.

Weipeng Zhang, received his B.E. and M.S degrees both from Department of Computer Science and Engineering, Shanghai Jiao Tong University. He is currently pursuing a Ph.D. in artificial intelligence, at the department of computer science and engineering, Shanghai Jiao Tong University. His research interests include machine learning and data mining.

Ning Zhang, received her Ph.D. from Institute of Automation, Chinese Academy of Science at 2016. Now she is doing research at department of computer science and engineering, Shanghai Jiao Tong University. The research interests include the sequence analysis and generation, multimedia understanding, reinforcement learning. She has published series of papers as first author on top journals and proceedings, including IJCAI, TNNLS, TIFS and so on.



Junchi Yan, is currently an Associate Professor with Department of Computer Science and Engineering, Shanghai Jiao Tong University. Before that, he was a Senior Research Staff Member and Principal Scientist for industrial vision with IBM Research where he started his career since April 2011. He obtained the Ph.D. at the Department of Electronic Engineering of Shanghai Jiao Tong University, China in 2015. Meanwhile he received the ACM China Doctoral Dissertation Nomination Award and China Computer Federation Doctoral Dissertation Award. His research interests are machine learning and visual computing. He serves as an Associate Editor for IEEE ACCESS, Managing Guest Editor for IEEE Transactions on Neural

Networks and Learning Systems, Pattern Recognition Letters, and on the executive board of ACM China Multimedia Chapter. He is the awardee of IBM Journal of Emphasis and a Member of IEEE.



Guofu Li, is the leading AI scientist of Pingan Asset Management Company. He received his Ph.D. in Computer Science and Informatics from University College Dublin in 2014. He was a lecturer of University of Shanghai for Science and Technology. His research interests include artificial intelligence, machine learning, ontology and natural language processing.

Xiaokang Yang, received the B.S. degree from Xiamen University, Xiamen, China, in 1994, the M.S. degree from the Chinese Academy of Sciences, Shanghai, China, in 1997, and the Ph.D. degree from Shanghai Jiao Tong University, Shanghai, in 2000. He has authored and coauthored over 200 refereed papers and holds 40 patents. His current research interests include visual signal processing and communication, media analysis and retrieval, and pattern recognition. He is an Associate Editor of the IEEE Transactions on Multimedia and a Senior Associate Editor of the IEEE Signal Processing Letters.